

Foundations of R Chapter 1

Robert Gulotty

9/1/2018

This is the first of 10 weeks of R materials that will guide you through the programming side of data science. While these worksheets are intended to be self contained, there are a couple prerequisites. You should visit datacamp.com and work through the intro and ideally intermediate classes in R. These will provide basic insight into the fundamentals of R, including objects in R and some insights into how functions work.

In these chapters R code are contained in highlighted boxes. Below you may see output, which are prefaced with `##`. In Rstudio, the suggested platform for using R, this code is typed out in a **script**, and then run in the console by selecting the code and typing `command+enter`. For more general help, type `help.start()` in the console and click enter. For the r-reference card/ cheat sheet, go to <https://cran.r-project.org/doc/contrib/Short-refcard.pdf>

This chapter is different than those that follow in that it is designed to get the reader from 0 to 100. Subsequent chapters will then dive into each command, but the goal for this chapter is to appreciate the advantages of using R for data analysis. As such, rather than start with simple objects like integers, vectors and functions, we begin with data and learn the dplyr approach to analyzing data, and then we cover plotting. These are advanced tools, but as you will see, even a novice can get usable and interesting output.

Installing packages

The first time you run R, you will have to install packages. These are “libraries” that contain interesting code and data for us to use. You install them by using a function called `install.packages`. The first thing inserted into the `install.packages` function is the name of the package. Below you will see code that installs ‘dplyr’, ‘ggplot2’ and ‘nycflights’. Notice, the packages are case sensitive and surrounded by quotation marks. The next thing listed within the parenthesis is generally omitted, but if specified, tells the computer to look for these packages in the rstudio repository. CRAN, the R organization, has many repository options, and if you leave off this part, R will use your default repository, set the first time you run a install command. The third part, setting dependencies to `TRUE`, asks R to redownload all of the assumed packages that the installed package requires.

```
install.packages("dplyr", repos="http://cran.rstudio.com/", dependencies = TRUE)
install.packages("ggplot2", repos="http://cran.rstudio.com/", dependencies = TRUE)
install.packages("nycflights13", repos="http://cran.rstudio.com/", dependencies = TRUE)
```

You will should only have to run each of these commands once and when you restart R it will still be downloaded and installed. Sometimes packages need to be updated, which basically involves installing them again. It is never a problem to re-run a command in R.

Loading packages

After installation, you can load the packages with the `library` command. This function just uses the name of the package, with or without quotation marks. Notice the warning after you load the dplyr package. When a package is loaded, it introduces a number of functions that may have the same name as functions already loaded in R. In this case, `filter` and `lag` were functions in the preloaded `stats` package, but we are

rewriting it with the `dplyr` `filter` and `lag` command. If you want to use a function with a redundant name, you can always use `::` to make sure it is coming from the right package. For example, `dplyr::filter()` or `stats::filter()`.

```
library(ggplot2)
library(nycflights13)
```

```
## Warning: package 'nycflights13' was built under R version 4.0.2
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

If you want to see all the functions that come with a package, check out the help file by putting a specific command in the regular help function `help(package="ggplot2")`. Online you can find many resources at the official CRAN website, which organizes packages by topic.

Data and Dataframes

Today we are going to work with an amazing dataset, `flights`, which describes all the flights out of New York in 2013. These data are in a dataframe called `flights`. Dataframes are tables (technically lists) of data stored as a named object. Dataframes that have special features to enable ease of integration with other R functions.

We begin by identifying the **dimensions** of the `flights` dataset, which is already loaded into R with the `nycflights` library. To do so we use the `dim` function. The `dim` function takes an R object and returns the number of dimensions of the dataframe.

```
dim(flights)
```

```
## [1] 336776      19
```

You will notice that the numbers are not labeled as rows and columns. A little heuristic, whenever dealing with a matrix or dataframe is that the order is Rows and Columns, or Rum and Coke, Roman Catholic. R x C. In this case, we have 336,776 rows, and 19 columns. The rows contain each observation, here flights, the columns refer to a **variable**, a characteristic or measured attribute of that flight.

When using data from libraries, you can use the `?` command, as follows : `?flights`

The most common tool that I use when I get a new dataset is the function called `head()`. This is a command line way to look at the top, or head, of a dataset.

```
head(flights)
```

```
## # A tibble: 6 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     517             515           2     830             819
## 2  2013     1     1     533             529           4     850             830
## 3  2013     1     1     542             540           2     923             850
## 4  2013     1     1     544             545          -1    1004            1022
## 5  2013     1     1     554             600          -6     812             837
## 6  2013     1     1     554             558          -4     740             728
## # ... with 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
# head(flights, n=8)
```

Note that you get the first 6 rows by default. You can change this by adding to the head command an additional parameter “n”. `head()` has a companion `tail()`, which by default looks at the bottom 6 rows.

R is a language of functions. The things within the `()` are the inputs to the function, or in R-speak, the **Arguments** of the function. The output of the function is called the **Value** of the function. We have now seen two special examples of functions, `library()` and `install.packages()`, as well as one standard function, `head()`. As with all things in programming, the spelling and capitalization matters. These should be eventually memorized, but if you forget how to spell something, try typing the beginning of it and hit the tab key to autocomplete.

Looking Inside the Data.Frame

The fundamental goal of the data scientist is to make predictions and draw inferences from data. Dataframes, like the flights dataframe, are the store containers for R. To analyze the data, we need to get the data out.

We can learn about the kinds of data in our dataframe by using the `str()` function. `str()` gives the structure of things in the environment.

```
str(flights)
```

```
## tibble [336,776 x 19] (S3: tbl_df/tbl/data.frame)
## $ year      : int [1:336776] 2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
## $ month     : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
## $ day       : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
## $ dep_time  : int [1:336776] 517 533 542 544 554 554 555 557 557 558 ...
## $ sched_dep_time: int [1:336776] 515 529 540 545 600 558 600 600 600 600 ...
## $ dep_delay : num [1:336776] 2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
## $ arr_time  : int [1:336776] 830 850 923 1004 812 740 913 709 838 753 ...
## $ sched_arr_time: int [1:336776] 819 830 850 1022 837 728 854 723 846 745 ...
## $ arr_delay : num [1:336776] 11 20 33 -18 -25 12 19 -14 -8 8 ...
## $ carrier   : chr [1:336776] "UA" "UA" "AA" "B6" ...
## $ flight    : int [1:336776] 1545 1714 1141 725 461 1696 507 5708 79 301 ...
## $ tailnum   : chr [1:336776] "N14228" "N24211" "N619AA" "N804JB" ...
## $ origin    : chr [1:336776] "EWR" "LGA" "JFK" "JFK" ...
## $ dest      : chr [1:336776] "IAH" "IAH" "MIA" "BQN" ...
## $ air_time  : num [1:336776] 227 227 160 183 116 150 158 53 140 138 ...
## $ distance  : num [1:336776] 1400 1416 1089 1576 762 ...
## $ hour      : num [1:336776] 5 5 5 5 6 5 6 6 6 6 ...
## $ minute    : num [1:336776] 15 29 40 45 0 58 0 0 0 0 ...
## $ time_hour : POSIXct[1:336776], format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" ...
```

You may notice the first line of output gives the “class” of flights. Here flights has three types, `tbl_df`, `tbl` and `data.frame`. Only the third is relevant for us today. It then lists the structure of the content of the data. R has several basic data types:

- **int** refers to integers, such as 22, -9 and 0.
 - **num** refers to real numbers, such as .03, 19, and π .
 - **chr** refers to characters strings, or letter based data, such as “Alabama”, “horseradish”, and “C-3PO”.
 - **POSIXct** is a date format, which we will learn about later.
 - **logi** (not listed here) refers to logical types, TRUE and FALSE, manipulated by the logical commands described in the next section.
 - **Factor** (not listed here) refers to a categorical variable, a special encoded character vector.
-

Manipulating data in dataframes.

There are several basic commands to analyze data in a dataframe. The first is to subset rows, which is accomplished with the command `filter`. `filter` takes on three arguments. The first is your data frame. After that, you put any number of logical conditions that you want to query from that dataset.

It is easiest to see in an example. Suppose we want to get the data for flights on January 1. We know from the codebook listed at `?flights` that the dataset contains a variable for the day and for the month, named `day` and `month` respectively. `filter` lets us name these variables and put conditions on them. Here we set the filter for days equal to 1 and months equal to 1.

```
filter(flights, month == 1, day == 1)
```

```
## # A tibble: 842 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     517           515           2     830           819
## 2  2013     1     1     533           529           4     850           830
## 3  2013     1     1     542           540           2     923           850
## 4  2013     1     1     544           545          -1    1004          1022
## 5  2013     1     1     554           600          -6     812           837
## 6  2013     1     1     554           558          -4     740           728
## 7  2013     1     1     555           600          -5     913           854
## 8  2013     1     1     557           600          -3     709           723
## 9  2013     1     1     557           600          -3     838           846
## 10 2013     1     1     558           600          -2     753           745
## # ... with 832 more rows, and 11 more variables: arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

The way to read this is “give me the rows of flights where month is equal to 1 and day is equal to 1. Notice, month and day were both integers, so we had to set their values equal to an integer. If we instead had a variable that was a character, we would have to put our query in quotes. For example, suppose we want New Year Day flights to O’Hare:

```
filter(flights, month == 1, day == 1, origin=="ORD")
```

```
## # A tibble: 0 x 19
## # ... with 19 variables: year <int>, month <int>, day <int>, dep_time <int>,
## #   sched_dep_time <int>, dep_delay <dbl>, arr_time <int>,
## #   sched_arr_time <int>, arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

`filter` commands can take on many kinds of logical conditions, not just `==`. In the following, we describe several examples.

Logical commands

The `filter` command uses logical functions to find rows. In this case we used the `==` function. Under the hood, `filter` looks for logical vectors, like `TRUE`, `TRUE`, `FALSE`. So, in order to use the `filter` function, we need a function which takes data from the flights dataset (the input) and gives as a value (the output) a logical vector.

There are a number of these logical functions in R. These, and all standard functions, can be found in help files by typing `?‘COMMAND’`.

- `==` refers to ‘equal to’. It returns `TRUE` if both sides are equal and not missing.

- `>=` and `<=` refer to 'greater than or equal to' and 'less than or equal to'. These compare numbers.
- `!=` means not equal to.
- `&` is a logical **and**
- `|` is the logical **or**
- `!` is the logical negation.

We can add and combine these logical statements. Suppose now that flights to Chicago can arrive at O'Hare or Midway. To capture the logical or we can use the `|`.

```
filter(flights, month == 1, day == 1, dest=="ORD" | dest=="MDW" )
```

```
## # A tibble: 56 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     554           558          -4     740           728
## 2  2013     1     1     558           600          -2     753           745
## 3  2013     1     1     608           600           8     807           735
## 4  2013     1     1     629           630          -1     824           810
## 5  2013     1     1     656           700          -4     854           850
## 6  2013     1     1     709           700           9     852           832
## 7  2013     1     1     715           713           2     911           850
## 8  2013     1     1     739           745          -6     918           930
## 9  2013     1     1     749           710          39     939           850
##10  2013     1     1     828           830          -2    1027          1012
## # ... with 46 more rows, and 11 more variables: arr_delay <dbl>, carrier <chr>,
## #   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

Practice Q. How many functions are in the above code?

Practice Q. How many flights traveled to Des Moines International from New York in August?

Practice Q. On what day did a flight go to Lexington, KY (LEX)?

At this point you should be able to answer the first four questions in the assignment section of this document.

OPTIONAL: Base R comparison - subsetting without Dplyr

In “base” or unmodified R, all this would be more cumbersome, with the following commands used to grab new year’s day flights.

```
januaryflights <- flights[, "month"] == 1
firstofmonthflight <- flights[, "day"] == 1
NewYearslogical <- januaryflights & firstofmonthflight
flights[NewYearslogical,]
```

```
## Warning: The `i` argument of `[`() can't be a matrix as of tibble 3.0.0.
## Convert to a vector.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_warnings()` to see where this warning was generated.
```

Although we will use the dplyr commands, it is useful to learn a bit about the R syntax here. As with all programming problems, the key is to break up this cumbersome command into its parts.

The first line uses the logical function ==, but on the left hand side, we are asking for just the month column of flights.

flights[] is just the whole flights dataset, and inside the square brackets are a comma. What do you think we get when we type flights[4 , 4] ?

Were you right? Try changing the numbers.

As you can see, we have a way to get elements out of the dataframe. What do you think we get when we write,

- flights[, "dest"]

or

- flights[, "day"]

Remember, rows and columns.

Now lets say we want the 4th entry under month, we could just write

- flights[4 , "month"]

All of this indexing assumes we know in advance what index we are looking for. In practical terms though, we want to be able to search through the data to find specific entries.

Using vectors in filters

It would be very cumbersome if we had to write a logical ‘or’ any time we were interested in finding a subset. For instance, the following code gets all of the days of the 2013 US Presidential Inauguration for DC and saves it in a new dataframe called DCflights for later analysis.

```
DCflights <- filter(flights,
  month == 1,
  day == 18 | day == 19 | day == 20 | day == 21,
  dest == "IAD" | dest == "DCA")
```

If we know that we want the days between 18 and 21, we can use a sequence command, : and the logical command ‘%in%’. Lets look at each in turn. The following are three ways to make a sequence.

```
seq(18,21, by=1)
```

```
## [1] 18 19 20 21
```

```
c(18, 19, 20, 21)
```

```
## [1] 18 19 20 21
```

```
18:21
```

```
## [1] 18 19 20 21
```

Given a vector, or set of numbers, the `%in%` command tells us whether a thing is in the vector.

```
12%in%18:21
```

```
## [1] FALSE
```

```
19%in%18:21
```

```
## [1] TRUE
```

These can be combined in useful ways:

```
12:20%in%18:21
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE
```

Using this principle, we can use the `%in%` command in our previous query.

```
DCflightsMethod2 <- filter(flights,  
  month == 1,  
  day %in% 18:21,  
  dest == "IAD" | dest == "DCA" )
```


Changing the data

Often we would like to transform the data in the data frame, perhaps subtracting the mean, dividing by the standard deviation, or even combining two variables. This is done in R with the `mutate` command. The following code makes a simple data frame, `diet`, and then adds a variable for the total cost:

```
diet <- data.frame(Lunch = c("sandwich", "hot dog", "pizza"),
                  Cost = c( 3.50, 2.00, 4.99),
                  Number_eaten = c(1, 3, 2))

mutate(diet, total_cost=Cost*Number_eaten)
```

```
##      Lunch Cost Number_eaten total_cost
## 1 sandwich 3.50             1        3.50
## 2 hot dog  2.00             3        6.00
## 3 pizza   4.99             2        9.98
```

We can construct new variables out of combinations of old variables, functions, or both:

```
mutate(diet,
       demeaned_cost=Cost-mean(Cost),
       ln_Number_eaten = log(Number_eaten)
)
```

```
##      Lunch Cost Number_eaten demeaned_cost ln_Number_eaten
## 1 sandwich 3.50             1    0.003333333    0.0000000
## 2 hot dog  2.00             3   -1.496666667    1.0986123
## 3 pizza   4.99             2    1.493333333    0.6931472
```

Notice we use the `=` for creating new variables inside the `mutate`, but `<-` for creating new variables outside the `mutate`.

Application: Weekend Flights

Suppose we want a new variable which indicates whether the flight occurs on a weekend in January.

The dataset does not have the day of the week, just the date. The key to any computational problem is to break it into smaller pieces. My strategy: According to iCal, January 1, 2013 was a Tuesday. Suppose we want to count from there by 7.

We create new variables using the `mutate` command. The following gets us whether a flight is on a Tuesday.

```
JanFlights <- filter(flights, month==1)

JanFlights <- mutate(JanFlights, DayisTue = day%in%seq(1, 31, by = 7))
```

To validate our strategy, let's count up the number of times the day is Tuesday. We can do that with the `count` command.

```
count(JanFlights, DayisTue)
```

```
## # A tibble: 2 x 2
##   DayisTue      n
##   <lg1>    <int>
## 1 FALSE    22589
## 2 TRUE     4415
```

Using this logic the following should give us the weekend.

```
JanFlights <- mutate(JanFlights,  
                      DayisWeekend = day%in%seq(5, 31, by = 7) | day%in%seq(6, 31, by = 7))  
count(JanFlights, DayisWeekend)
```

```
## # A tibble: 2 x 2  
##   DayisWeekend     n  
##   <lgl>         <int>  
## 1 FALSE       20971  
## 2 TRUE        6033
```

This last command is more complicated than the beginning, but its constituent parts were built up from smaller pieces.

Summarising Data

Within these DC flights, suppose we want to get the longest someone traveled direct to see the inauguration. First we need to know what variable gives us the travel distance. Here we will use the air time variable. The maximum is a **summary statistic** of the data, in the sense that the answer depend on knowing the value of every row of the data. This is done in R with the summarise command.

The summarise command takes on a dataframe and returns a smaller dataframe. The syntax is very similar to that of mutate and filter. Note here the max command need a second entry to tell it to ignore missing data. Some flights have missing air_time.

```
summarise(DCflights, max_air=max(air_time, na.rm=T))
```

```
## # A tibble: 1 x 1
##   max_air
##   <dbl>
## 1      61
```

It seems that the max is 61. We can pull out the rows where that is the case using the filter command:

```
filter( DCflights, air_time==61)
```

```
## # A tibble: 1 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>      <dbl>    <int>         <int>
## 1  2013     1    20    1421           1429        -8     1552           1547
## # ... with 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

mutate, filter, and summarise are each dplyr commands. There are a few more to learn, but first, we will learn how to explore data visually.

#Advanced Plotting

After you have mastered the above commands, we can turn to advanced plotting techniques, from **ggplot2**, the most cutting edge data visualization software on the market. Learning about ggplot2 will be a substantial part of this course, and a great tool you will use for the rest of your careers.

ggplot2 has a particular philosophy about how to plot. The basic parts are a **ggplot()** command to input the data, and then layers or **geoms** are added with **+** signs afterwards.

Inside a ggplot function we start with some data, and then we tell the computer which rows and columns from the data are to be used as the x and y axis. This is done inside a function called an aesthetic command **aes()**. The neat thing is that just like mutate and filter, the aesthetic command does not need quotation marks, more on that later.

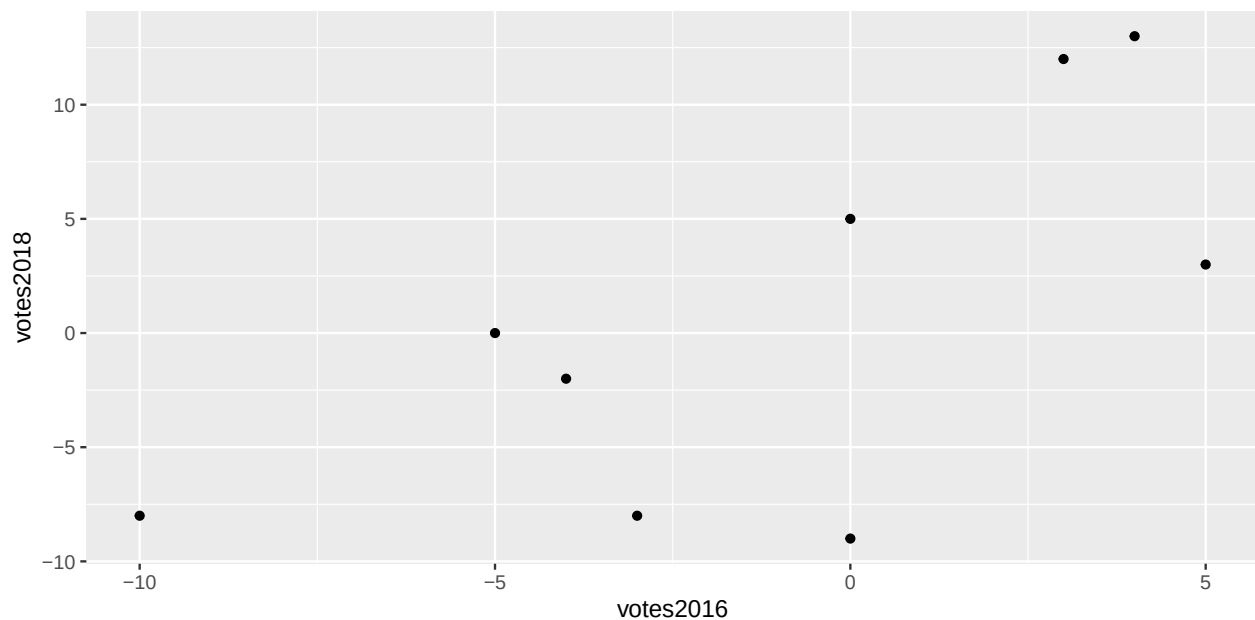
The next thing to do is add a layer, or **geom**, which tells **ggplot** what shape you want the data to take. Here is a list of the ones I use most often: **+ geom_points** : Common scatterplot. Takes on a **x** and **y** **aes**. **+ geom_bar** : Barplot, great for categorical data. **+ geom_histogram** : A histogram, which shows the proportion or frequency of a continuous variable in various bins. **+ geom_density** : Displays a smoothed histogram (essentially). **+ geom_smooth** : Adds a regression line to a scatterplot. Defaults to a smoothed local regression. **+ geom_text** : like a scatterplot but with text.

These commands will show up throughout the course.

We start with a very simple dataframe and plot:

```
votes2016<-c(-5, -4, -3, -10, 0, 3, 4, 0, 5)
votes2018<-c(0, -2, -8, -8, -9, 12, 13, 5, 3)
df<-data.frame(votes2016=votes2016, votes2018=votes2018)

ggplot(df, aes(x = votes2016, y = votes2018)) +
  geom_point()
```



Simple, clear, attractive. As we will see in the following, these techniques can be combined to perform powerful analysis

Flight time analysis

Many people prefer shorter flights to longer ones. There is a variable called `air_time`, but suppose I were worried about the extra time on the tarmac as well. To do so, we will add a new variable to the dataset, `flighttime`, by subtracting `dep_time` from `arr_time`. We will add that variable using the `mutate()` function. Again, remember you use an `=` then the content of the new variable.

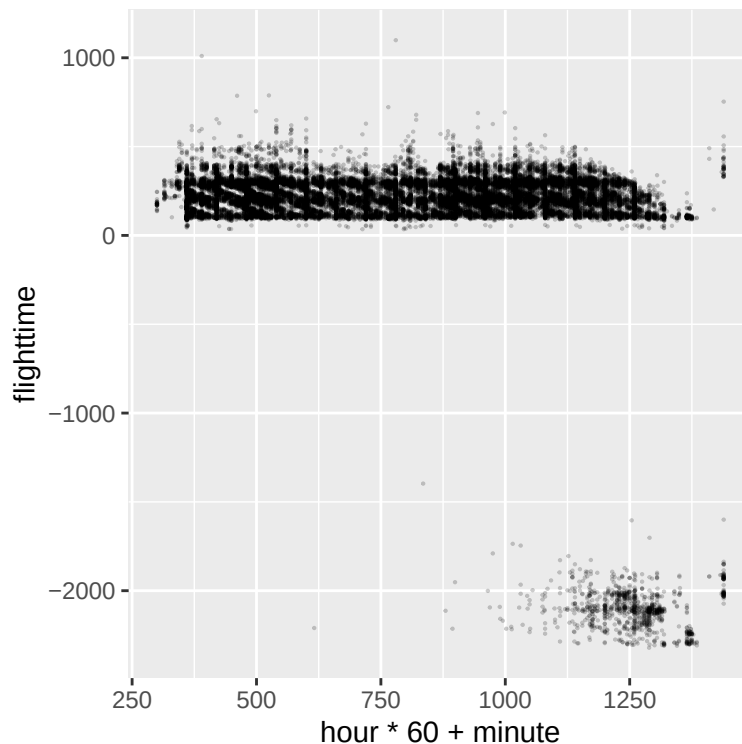
```
flights <- mutate(flights, flighttime = arr_time - dep_time)
```

Step 1 of any analysis is to visualize the data. Here we use a simple plot to visualize flights. Remember you can learn about all commands using `?qplot`. Now the `flights` data is a bit too large to plot all of it, so let's create a subsample. Then we can plot the flight time against the start time.

```
flights_s <- sample_frac(flights, .1)
```

```
ggplot(data = flights_s, aes(x = hour*60+minute, y = flighttime)) + geom_point(alpha=.2, size=.1)
```

```
## Warning: Removed 904 rows containing missing values (geom_point).
```



Immediately we see that something is wrong. Flight time should not be negative. Our measure has some flights spilling over into the next day. A quick solution is to add 2400 hours when the flighttime is negative. We can do that with an `ifelse` command. `ifelse` commands take a logical statement and return the second element if it is true and the third element if it is false.

```
ifelse(2==3, "Is correct", "Is not correct")
```

```
## [1] "Is not correct"
```

`ifelse` has a multicategory cousin, `case_when`, which can be used for recoding variables into more than two categories.

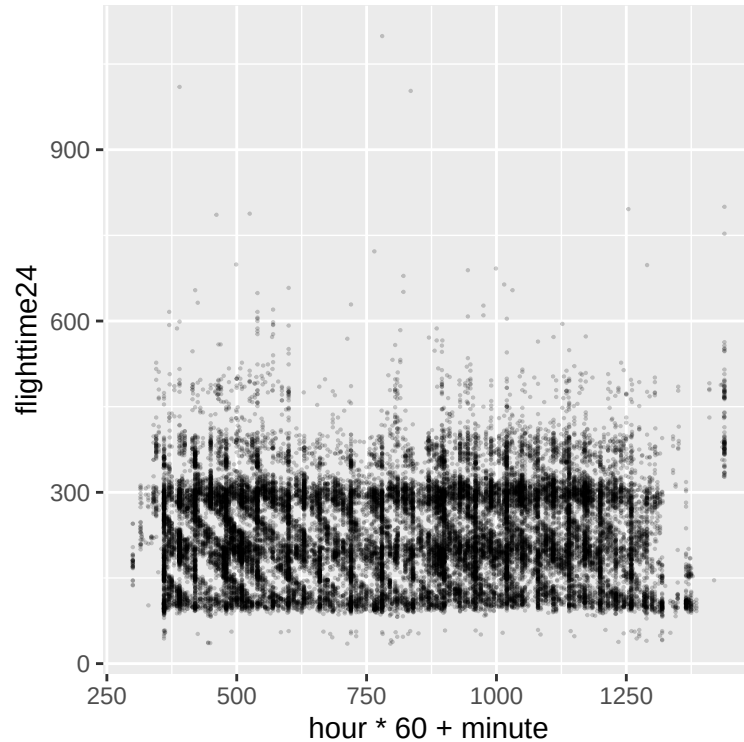
```
flights_s <- mutate(flights_s,  
                    flighttime = arr_time - dep_time,
```

```

    flighttime24 = ifelse( flighttime < 0, flighttime+2400, flighttime))
ggplot(data = flights_s, aes(x = hour*60+minute, y = flighttime24) )+geom_point(alpha=.2, size=.1)

## Warning: Removed 904 rows containing missing values (geom_point).

```



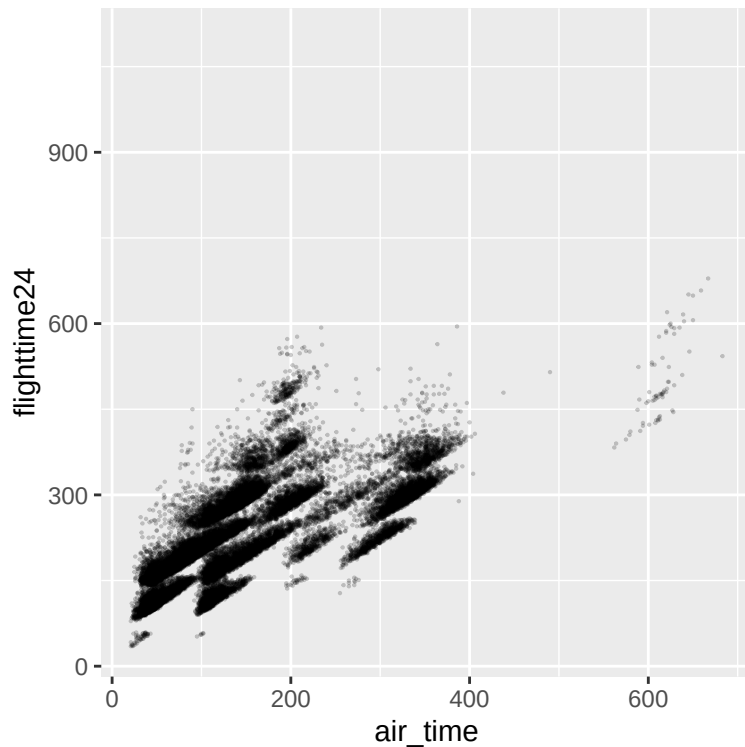
Now we see that flight times range in reasonable amounts. Let's look more carefully at the distribution of flighttime24 and compare it to the variable for air_time.

```

ggplot(data = flights_s, aes(x = air_time, y = flighttime24) )+geom_point(alpha=.2, size=.1)

## Warning: Removed 993 rows containing missing values (geom_point).

```



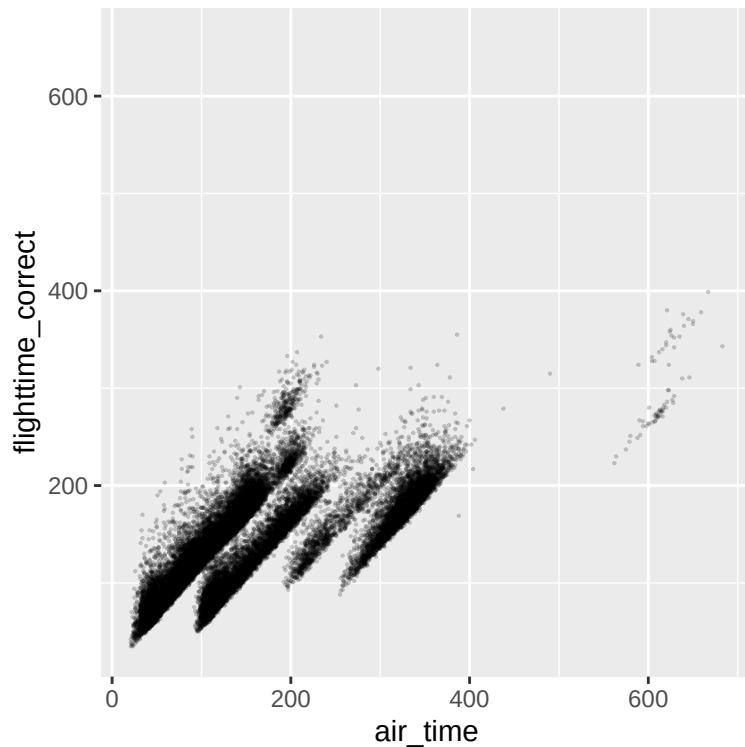
The data appears to arrive in little pyramids. This is because time is not a number, but rather a combination of an hour indicator and a minute indicator, we are getting off when we subtract. This all shows that we need to be careful in interpreting variables.

To fix the data, we can divide `dep_time` and `arr_time` into their respective hour and minute components, multiply appropriately, and then take the difference. Here we use a trick of dividing and then dropping the decimal points using the `floor` command, and adding to that a modulus or `%%` command to get the remainder after dividing by 100.

```
flights_s <- mutate(flights_s,
                    arr_time_mins= floor(arr_time/100)*60+arr_time%%100,
                    dep_time_mins= floor(dep_time/100)*60+dep_time%%100,
                    arr_time_mins = ifelse( arr_time_mins-dep_time_mins < 0, arr_time_mins+2, arr_time_mins-dep_time_mins),
                    flighttime_correct=arr_time_mins-dep_time_mins)

ggplot(data = flights_s, aes(x = air_time, y = flighttime_correct) )+
  geom_point(alpha=.2, size=.1)
```

```
## Warning: Removed 993 rows containing missing values (geom_point).
```



It still isn't smooth. What is going on? Why are flights jumping by an hour? What is it about certain flights that push them to an earlier hour? You might have guessed. We need the time zone. Incorporating the time zone, a variable available in the airports data frame, will require learning about joins, a topic we will get to in later chapters.

Recap

This chapter introduced several useful data manipulation commands, `filter`, `mutate`, as well as several useful logical commands. In the following chapters, we will return to the subject of plotting as well as the other data manipulation commands in dplyr: `select`, `group_by`, `summarise` and `arrange`. Even with the few commands that we have covered, it should be clear that R is a powerful tool for data manipulation. In the next chapter, we finish our brief introduction to the power of R as a **statistical** package.

Problems

Package Loading

```
library(dplyr, warn.conflicts = FALSE)
library(ggplot2)
library(nycflights13)
```

- (1) What are the types of data that serve as the argument and value of the following functions?
 - `nchar()`
 - `log()`
 - `paste()`
 - `is.na()`
- (2) What was the longest physical distance taken out of New York in 2013?
- (3) In R, we can use scientific notation. `2e3` is 2000. Why is `log(1e10)-log(1e8)` the same as `log(1e2)`?
- (4) Without running these commands, what are the values of C, D, and E?

```
A <- "Iowa"
B <- "Kansas"

C <- !((A == "Kentucky") | (B == "Kansas"))

D <- ((nchar(A) > 2) & is.na(B)) | (!is.na(A))

E <- ifelse( C, D, A)
```